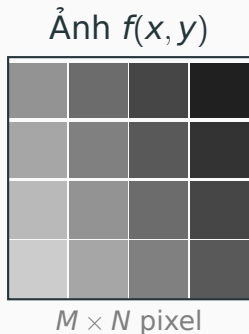


Biến đổi Fourier 2D

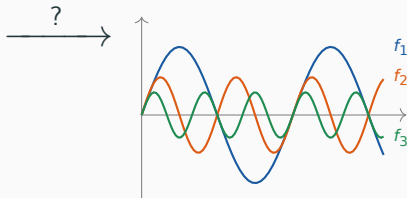
Tuần 3 — Xử lý ảnh & Thị giác máy tính

ĐH GTVT TP.HCM · Môn 121036

Câu hỏi khởi động



Tổ hợp sóng?



Có thể biểu diễn ảnh $f(x, y)$ như một **tổng của các sóng tuần hoàn** không?

Định nghĩa: DFT 2D

Biến đổi Fourier rời rạc 2D

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \underbrace{e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}}_{\text{hàm cơ sở tại } (u,v)}$$

với $u = 0, \dots, M-1$ và $v = 0, \dots, N-1$.

Đầu vào: ảnh $f(x, y) \in \mathbb{R}^{M \times N}$

Đầu ra: ma trận $F(u, v) \in \mathbb{C}^{M \times N}$

Mỗi $F(u, v)$ là một **tích vô hướng**: đo mức độ “khớp” của f với sóng có tần số (u, v) .

Hàm cơ sở của DFT là gì?

Hàm cơ sở tại tần số (u, v) :

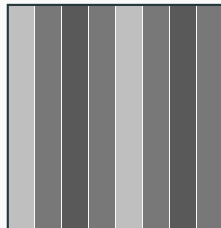
$$\phi_{u,v}(x, y) = e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Dạng Euler:

$$= \cos\left(2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)\right) + j\sin\left(2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)\right)$$

- Đây là một **sóng phẳng 2D**
- Dao động theo hướng $(u/M, v/N)$
- u chu kỳ theo chiều x , v chu kỳ theo chiều y

$\text{Re}(\phi_{u,0})$ — sóng ngang



1 chu kỳ

Tính trực giao của các hàm cơ sở

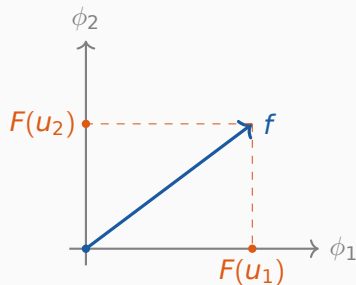
$$\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \phi_{u,v}(x,y) \overline{\phi_{u',v'}(x,y)} = MN \cdot \delta(u - u') \delta(v - v')$$

Ý nghĩa:

- Hai hàm cơ sở khác tần số: tích vô hướng = 0
- Cùng tần số: tích vô hướng = MN

⇒ Tập $\{\phi_{u,v}\}$ là **cơ sở trực giao** của $\mathbb{C}^{MN}$.

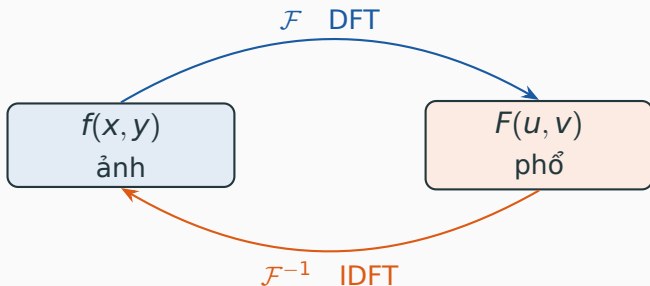
⇒ DFT là phép **chiếu** f lên từng hướng cơ sở — giống như chiếu vector lên các trục tọa độ.



Biến đổi Fourier nghịch (IDFT)

Tái tạo ảnh từ phổ Fourier

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$



Phổ biên độ

Từ $F(u, v) \in \mathbb{C}$, tách ra:

$$|F(u, v)| = \sqrt{\text{Re}(F)^2 + \text{Im}(F)^2}$$

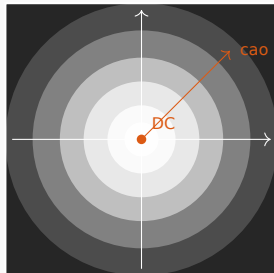
Hiển thị thực tế (nén dải động):

$$S(u, v) = \log(1 + |F(u, v)|)$$

Ý nghĩa: $|F(u, v)|$ lớn

$\Rightarrow$ tần số (u, v) đóng góp *nhiều* vào ảnh.

Phổ biên độ $|F|$ (log)



Phổ pha

$$\angle F(u, v) = \arctan \frac{\text{Im}(F)}{\text{Re}(F)} \in [-\pi, \pi]$$

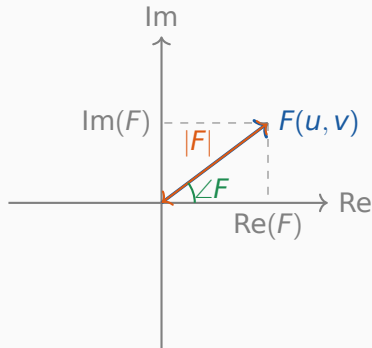
Pha mang thông tin vị trí
của các cấu trúc trong ảnh.

Quan sát thực nghiệm nổi tiếng

Hoán đổi pha của hai ảnh A và B:

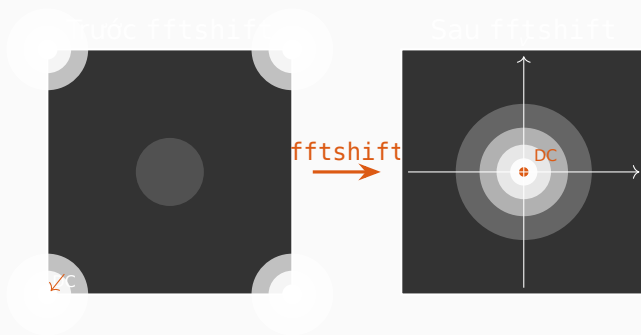
ảnh tái tạo trông giống ảnh có pha
hơn ảnh có biên độ.

⇒ **Pha quan trọng hơn biên độ**
để tái tạo ảnh trung thực.

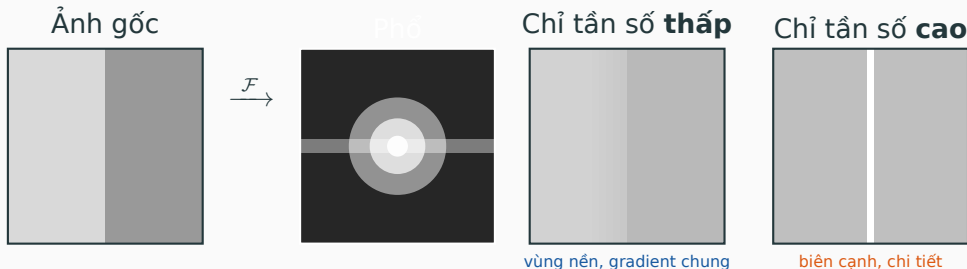


fftshift: Đưa tần số thấp về trung tâm

DFT chuẩn đặt tần số $u = 0, v = 0$ (DC) tại **góc trên trái**.
`np.fft.fftshift` dịch để DC về **trung tâm**.



Tần số thấp và tần số cao trong ảnh

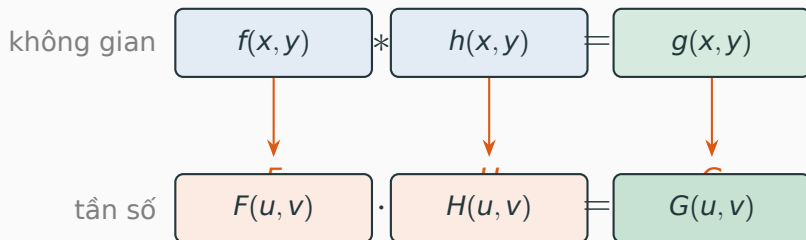


Định lý tích chập

Định lý tích chập (Convolution Theorem)

$$f(x, y) * h(x, y) \xrightarrow{\mathcal{F}} F(u, v) \cdot H(u, v)$$

Tích chập trong miền **không gian** $\equiv$ nhân điểm-điểm trong miền **tần số**.



Chứng minh Định lý tích chập

Bắt đầu từ định nghĩa tích chập và DFT:

$$\mathcal{F}\{f * h\}(u, v) = \sum_{x, y} \underbrace{\left[\sum_{s, t} f(s, t) h(x-s, y-t) \right]}_{(f * h)(x, y)} e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

Đổi thứ tự tổng; đặt $x' = x - s$, $y' = y - t$:

$$= \sum_{s, t} f(s, t) e^{-j2\pi \left(\frac{us}{M} + \frac{vt}{N} \right)} \cdot \sum_{x', y'} h(x', y') e^{-j2\pi \left(\frac{ux'}{M} + \frac{vy'}{N} \right)}$$

$$= \underbrace{F(u, v)}_{\mathcal{F}\{f\}} \cdot \underbrace{H(u, v)}_{\mathcal{F}\{h\}} \quad \blacksquare$$

Lọc trong miền tần số: Lọc thấp (Low-pass)

$$G(u, v) = H_{LP}(u, v) \cdot F(u, v) \xrightarrow{\mathcal{F}^{-1}} g(x, y)$$

Lọc thấp lý tưởng:

$$H_{LP}(u, v) = \begin{cases} 1 & D(u, v) \leq D_0 \\ 0 & D(u, v) > D_0 \end{cases}$$

với $D(u, v) = \sqrt{u^2 + v^2}$ (sau fftshift).

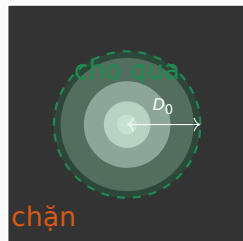
Tác dụng: giữ lại tần số thấp

⇒ ảnh bị làm mờ.

Nhược điểm: bộ lọc bước nhảy gián đoạn

⇒ **hiện tượng Gibbs (ringing)**.

H_{LP} trong phổ

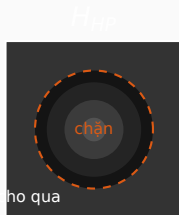


Lọc cao và Lọc Gaussian

Lọc cao (High-pass):

$$H_{HP}(u, v) = 1 - H_{LP}(u, v)$$

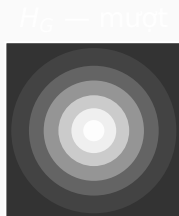
Giữ lại tần số cao — tăng cường biên cạnh.



Lọc Gaussian (Low-pass):

$$H_G(u, v) = e^{-\frac{u^2+v^2}{2\sigma_f^2}}$$

Không có bước nhảy $\Rightarrow$ **không ringing.**



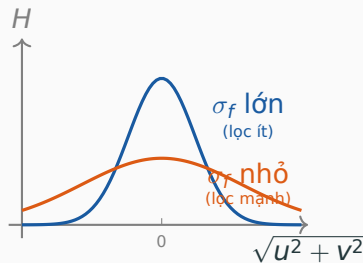
Gaussian: Mối quan hệ σ và σ_f

Trong miền không gian:

$$h(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Biến đổi Fourier của Gaussian là Gaussian:

$$H(u, v) = e^{-\frac{u^2+v^2}{2\sigma_f^2}}, \quad \sigma_f = \frac{1}{2\pi\sigma}$$



Quan hệ nghịch đảo

$$\sigma \uparrow \Rightarrow \sigma_f \downarrow$$

Kernel rộng hơn = lọc thấp chặt hơn = mờ nhiều hơn.

Thuật toán FFT: Chia để trị

DFT ngây thơ cho ảnh $N \times N$: $O(N^4)$ phép nhân.

Ý tưởng FFT (Cooley-Tukey, 1965): tách DFT N thành hai DFT $N/2$:

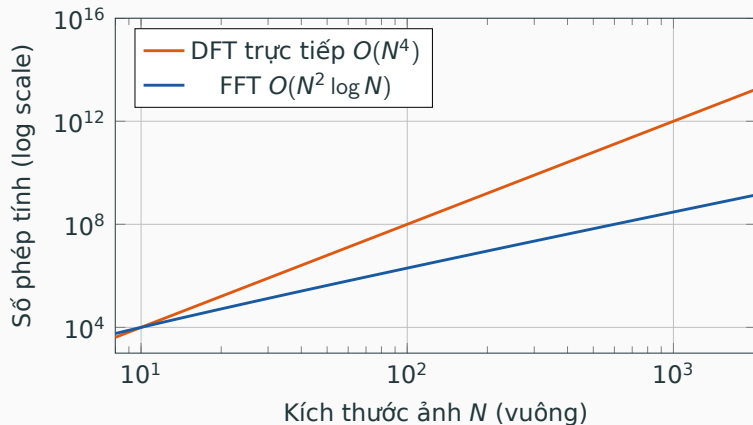
$$F(u) = \underbrace{\sum_{x \text{ chẵn}} f(x) e^{-j2\pi ux/(N/2)}}_{E(u)} + e^{-j2\pi u/N} \underbrace{\sum_{x \text{ lẻ}} f(x) e^{-j2\pi ux/(N/2)}}_{O(u)}$$

Công thức đệ quy: $T(N) = 2T(N/2) + O(N) \Rightarrow T(N) = O(N \log N)$.

2D FFT: tách theo hàng và cột

Áp FFT 1D lên từng hàng $\rightarrow$ áp FFT 1D lên từng cột $\Rightarrow O(MN \log MN)$

So sánh độ phức tạp: DFT trực tiếp vs FFT



Với $N = 1024$: DFT $\approx 10^{12}$ phép tính, FFT $\approx 10^7$ — **khác nhau 10^5 lần.**

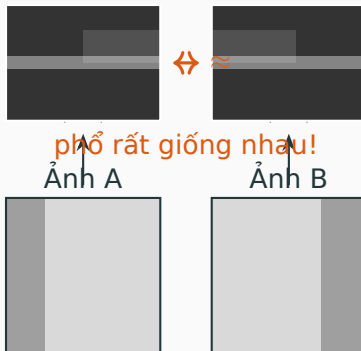
Các tính chất quan trọng của DFT 2D

Tính chất	Miền không gian	Miền tần số
Tuyến tính	$af + bg$	$aF + bG$
Dịch chuyển không gian	$f(x-x_0, y-y_0)$	$F(u, v) e^{-j2\pi(\frac{ux_0}{M} + \frac{vy_0}{N})}$
Tích chập	$f * h$	$F \cdot H$
Định lý Parseval	$\sum_{x,y} f ^2$	$\frac{1}{MN} \sum_{u,v} F ^2$
Đối xứng Hermitian	f thực	$F(-u, -v) = F^*(u, v)$

Đối xứng Hermitian: nếu f thực thì chỉ cần lưu *một nửa* phổ.

Hạn chế cốt lõi của DFT

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\dots)} \quad \leftarrow \text{tổng trên **toàn bộ** ảnh}$$



★ Ý tưởng cốt lõi

Tích chập trong miền không gian $\equiv$
nhân điểm-điểm trong miền tần số:

$$f * h \quad \mathcal{F} \quad F \cdot H$$

Mọi bộ lọc tuyến tính đều là một
phép nhân $H \cdot F$ trong miền Fourier.

FFT thực hiện phép nhân đó trong $O(MN \log MN)$ thay vì $O(M^2N^2)$.

Những gì sẽ thực hiện trong notebook:

1. Tính `fft2` trên ảnh tổng hợp; hiển thị phổ log sau `fftshift`
2. Xây dựng H_{LP} hình tròn; nhân $H_{LP} \cdot F$; IDFT
— quan sát ringing (hiện tượng Gibbs)
3. Thay bằng H_G Gaussian — ringing biến mất
4. Xây dựng $H_{HP} = 1 - H_{LP}$; IDFT — chỉ còn biên cạnh
5. Kiểm tra: lọc Gaussian trong tần số $\equiv$ tích chập Gaussian trong không gian
(`np.allclose` trên cả hai kết quả)

Biến đổi Wavelet

Tuần 4 — Xử lý ảnh & Thị giác máy tính

ĐH GTVT TP.HCM · Môn 121036

Nhắc lại: Hạn chế của DFT

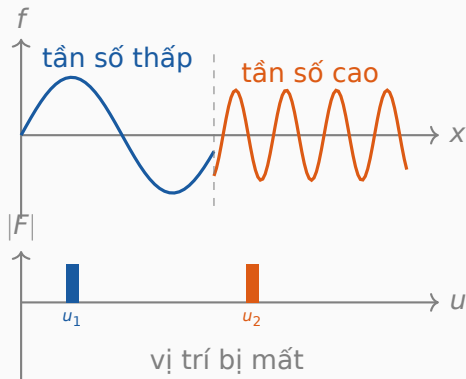
Hệ số Fourier:

$$F(u, v) = \sum_{x,y} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Mỗi $F(u, v)$ phụ thuộc **toàn bộ** ảnh.

Vấn đề

DFT không thể cho biết
tần số (u, v) **xuất hiện ở đâu**.



Ý tưởng Wavelet: Hàm cơ sở cục bộ

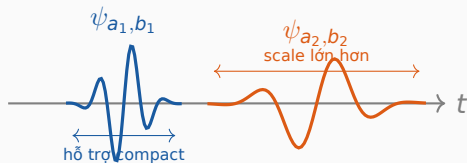
DFT dùng hàm cơ sở **toàn cục**:

$$e^{j2\pi ux/N} \leftarrow \text{trải dài vô hạn}$$

Wavelet dùng hàm cơ sở **cục bộ**:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \psi\left(\frac{t-b}{a}\right)$$

- $a > 0$: **co giãn** (scale) — kiểm soát tần số
 - $b \in \mathbb{R}$: **dịch chuyển** — kiểm soát vị trí
- $\Rightarrow$ mỗi hệ số wavelet mang *cả hai* thông tin: **tần số** và **vị trí**.



Wavelet mẹ ψ : Các điều kiện cần thiết

Từ **wavelet mẹ** $\psi(t)$, sinh ra toàn bộ họ hàm $\psi_{a,b}$.

Hai điều kiện bắt buộc:

(1) Trung bình bằng 0:

$$\int_{-\infty}^{+\infty} \psi(t) dt = 0$$

Đảm bảo wavelet là một *dao động*
— không phải hàm trung bình.
 $\Rightarrow$ bộ lọc **cao** (high-pass).

(2) Năng lượng hữu hạn:

$$\int_{-\infty}^{+\infty} |\psi(t)|^2 dt = 1$$

Chuẩn hoá để các hệ số wavelet
có thể so sánh được qua các scale.

Điều kiện admissibility (cần để tái tạo hoàn hảo):

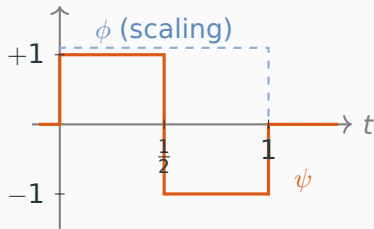
$$C_\psi = \int_0^\infty \frac{|\hat{\psi}(\omega)|^2}{\omega} d\omega < \infty$$

với $\hat{\psi}$ là biến đổi Fourier của ψ

Haar Wavelet: Định nghĩa

Wavelet mẹ Haar:

$$\psi^{\text{Haar}}(t) = \begin{cases} +1 & 0 \leq t < \frac{1}{2} \\ -1 & \frac{1}{2} \leq t < 1 \\ 0 & \text{còn lại} \end{cases}$$



Hàm tỉ lệ (scaling function):

$$\phi^{\text{Haar}}(t) = \begin{cases} 1 & 0 \leq t < 1 \\ 0 & \text{còn lại} \end{cases}$$

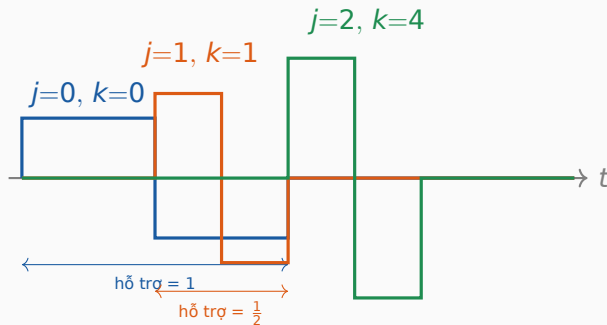
(ϕ tương ứng với bộ lọc **thấp**)

Haar Wavelet: Co giãn và Dịch chuyển

Sinh họ hàm từ ψ^{Haar} :

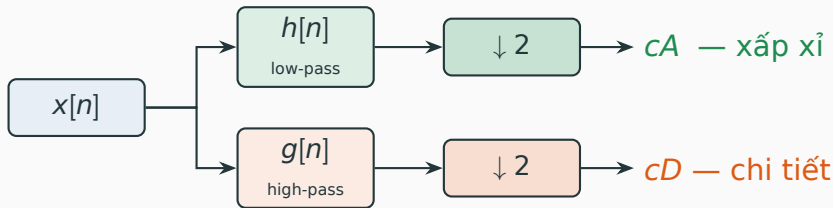
$$\psi_{j,k}(t) = 2^{j/2} \psi^{\text{Haar}}(2^j t - k), \quad j, k \in \mathbb{Z}$$

(j điều khiển **scale**, k điều khiển **vị trí**)



DWT 1D: Bộ lọc băng con (Subband Filterbank)

Trong thực tế, DWT rời rạc thực hiện qua **bộ lọc băng con**:



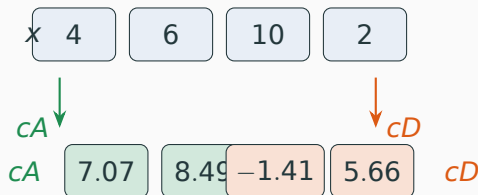
Với Haar:

$$h = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \quad (\text{trung bình}), \quad g = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix} \quad (\text{hiệu})$$

Sau hạ mẫu $\downarrow 2$: mỗi nhánh có $N/2$ mẫu. Tổng: $N/2 + N/2 = N$ — **bảo toàn số mẫu**.

Ví dụ số: Haar DWT 1 tầng

Cho $x = [4, 6, 10, 2]$. Áp Haar DWT:



$$cA[k] = \frac{x[2k] + x[2k + 1]}{\sqrt{2}}$$

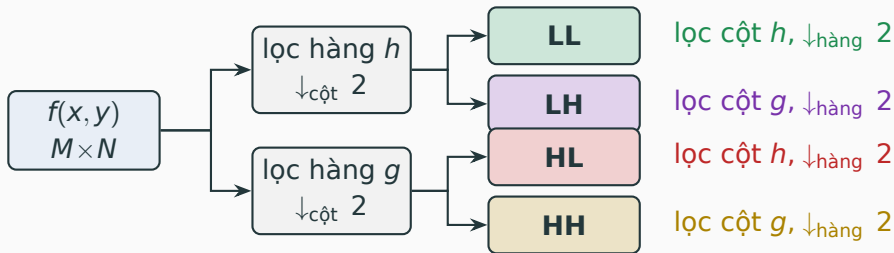
cA nắm bắt **xu hướng chung** của từng cặp.

$$cD[k] = \frac{x[2k] - x[2k + 1]}{\sqrt{2}}$$

$|cD[1]| = 5.66$ lớn $\Rightarrow$ có **biên** giữa $x[2] = 10$ và $x[3] = 2$.

DWT 2D: Lọc Hàng rồi Cột

DWT 2D áp dụng DWT 1D **tách biệt** theo từng chiều:



Kết quả: 4 subband, mỗi cái kích thước $\frac{M}{2} \times \frac{N}{2}$. Tổng: $4 \cdot \frac{MN}{4} = MN$ —
bảo toàn.

Bốn Subband: Ý nghĩa

	Low-cột	High-cột
Low-hàng	LL	LH
High-hàng	HL	HH

Band	Bộ lọc	Nội dung
LL	$H_r * H_c$	Xấp xỉ (mờ + hạ mẫu)
LH	$H_r * G_c$	Biên ngang
HL	$G_r * H_c$	Biên dọc
HH	$G_r * G_c$	Biên chéo , góc

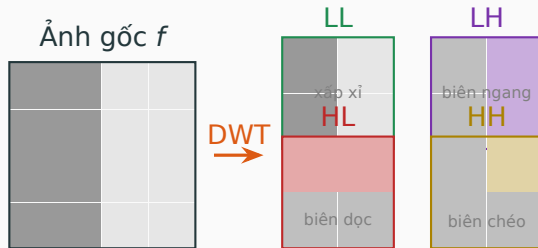
Lưu ý về LH:

Low theo hàng = mịn theo hàng.

High theo cột = thay đổi mạnh giữa các cột.

⇒ phát hiện **cạnh nằm ngang**.

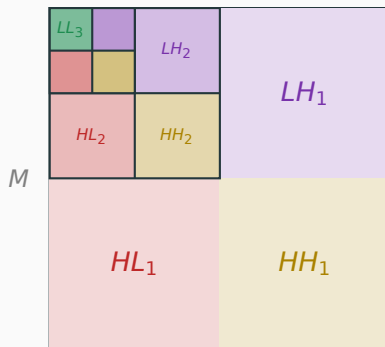
Minh họa: Subband trên ảnh đơn giản



Phân tích Đa tầng

Subband **LL** là một ảnh nhỏ hơn — có thể tiếp tục phân tích.

Tầng k : $LL_{k-1} \xrightarrow{\text{DWT}} LL_k, LH_k, HL_k, HH_k$ kích thước $\frac{M}{2^k} \times \frac{N}{2^k}$



Ứng dụng: Khử nhiễu — Quan sát cơ bản

Tín hiệu thực (cạnh, kết cấu)

⇒ hệ số wavelet **lớn**, tập trung tại ít vị trí.

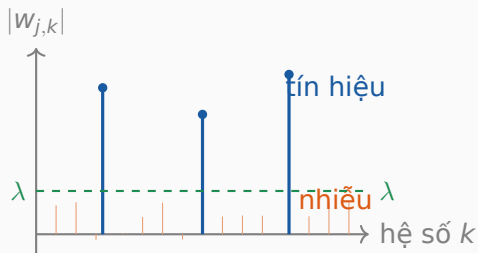
Nhiều ngẫu nhiên

⇒ hệ số wavelet **nhỏ**, phân tán đều khắp tất cả subband.

Chiến lược

Nếu $|w_{j,k}|$ nhỏ: **đặt về 0** (có thể là nhiễu).

Nếu $|w_{j,k}|$ lớn: **giữ lại** (có thể là tín hiệu).



Soft Thresholding

Hàm soft thresholding với ngưỡng λ :

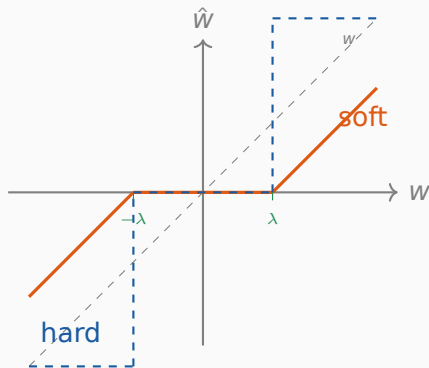
$$\hat{w} = \text{sign}(w) \cdot \max(|w| - \lambda, 0)$$

Tương đương:

$$\hat{w} = \begin{cases} w - \lambda & w > +\lambda \\ 0 & |w| \leq \lambda \\ w + \lambda & w < -\lambda \end{cases}$$

Hard thresholding (so sánh):

$$\hat{w}_{\text{hard}} = w \cdot \mathbf{1}[|w| > \lambda]$$



Bước nhảy tại $\pm\lambda \Rightarrow$ ringing sau tái tạo.

Soft: liên tục $\Rightarrow$ mượt hơn.

Chọn Ngưỡng: VisuShrink

Donoho & Johnstone (1994) đề xuất ngưỡng tối ưu:

$$\lambda^* = \hat{\sigma} \sqrt{2 \ln N}$$

Ước lượng $\hat{\sigma}$ từ subband HH_1 :

$$\hat{\sigma} = \frac{\text{median}(|c_{HH_1}|)}{0.6745}$$

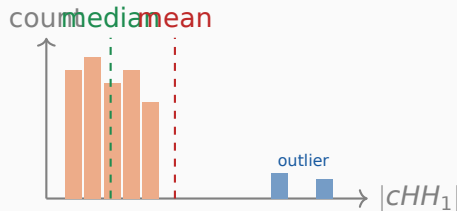
(0.6745 = trung vị của $|\mathcal{N}(0, 1)|$)

Tại sao dùng HH_1 ?

HH_1 chứa chi tiết cao nhất.

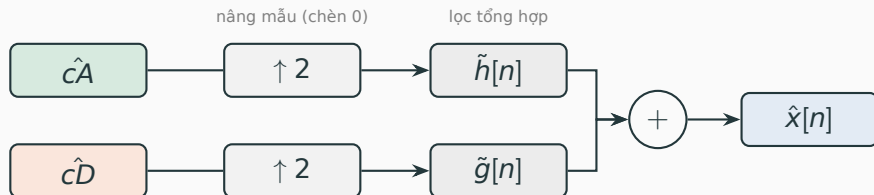
Nếu ảnh không có nhiều texture chéo,
 $HH_1 \approx$ thuần nhiễu.

Tại sao dùng median?



Median bền vững với outlier (hệ số cạnh thật).

Tái tạo: Biến đổi Wavelet Nghịch (IDWT)

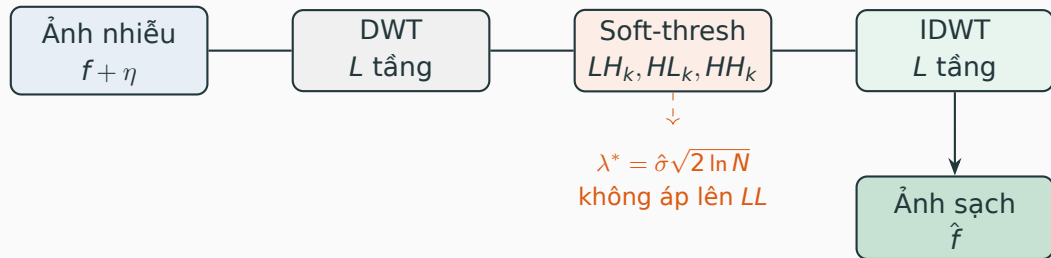


Điều kiện tái tạo hoàn hảo (Perfect Reconstruction):

$$\tilde{H}(z)H(z) + \tilde{G}(z)G(z) = 2z^{-k}$$

Với wavelet trực giao (như Haar, Daubechies): $\tilde{h}[n] = h[-n]$ và $\tilde{g}[n] = g[-n]$ — bộ lọc tổng hợp là **lật thời gian** của bộ phân tích.

Quy trình Khử Nhiễu Đầy đủ



Các trường hợp thất bại:

- λ quá lớn: xóa cả hệ số tín hiệu $\Rightarrow$ ảnh bị mờ, mất biên
- λ quá nhỏ: nhiễu còn sót lại
- Dùng Haar trên vùng mịn: hiện tượng **blocking** do bộ lọc hình chữ nhật

So sánh: DFT và DWT

	DFT	DWT
Hàm cơ sở	Sóng phẳng toàn cục	Wavelet cục bộ
Thông tin tần số	Có	Có
Thông tin vị trí	Không	Có
Phân tích đa thang	Không	Có
Phù hợp với	Texture đều, tuần hoàn	Biên, cấu trúc cục bộ
Khử nhiễu	Ringing (Gibbs)	Bảo toàn biên
Thư viện	<code>np.fft.fft2</code>	<code>pywt.dwt2</code>

★ Ý tưởng cốt lõi

DFT dùng hàm cơ sở **toàn cục** —
mỗi hệ số phụ thuộc toàn bộ ảnh.

DWT dùng hàm cơ sở **cục bộ** — mỗi
hệ số chỉ phụ thuộc một vùng nhỏ.

Bốn subband *LL*, *LH*, *HL*, *HH* phân tách thông tin xấp xỉ và
thông tin biên theo từng hướng, tại từng thang phân giải.

Demo: DWT và Khử nhiễu trong Python

Những gì sẽ thực hiện trong notebook:

1. Xây dựng ảnh tổng hợp có nhiễu Gaussian ($\sigma_\eta = 25$) bằng NumPy
2. Gọi `pywt.dwt2(img, 'db4', level=2)` — hiển thị 4 subband
3. Xác định thủ công: subband nào chứa biên? Subband nào chủ yếu là nhiễu?
4. Tính $\hat{\sigma}$ từ HH_1 ; tính $\lambda^* = \hat{\sigma}\sqrt{2 \ln N}$
5. Áp `pywt.threshold` lên LH_k, HL_k, HH_k (không áp lên LL)
6. Tái tạo bằng `pywt.idwt2`; so sánh PSNR trước và sau

Đặc trưng cục bộ là gì?

Tuần 3 — Phát hiện và mô tả đặc trưng

ĐH GTVT TP.HCM · Môn 121036

Bài toán: Ghép hai ảnh chụp cùng cảnh

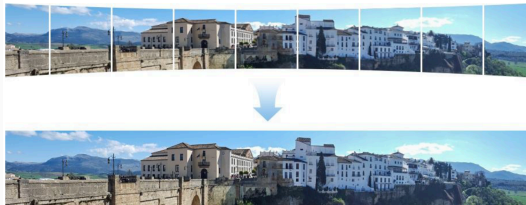


Figure 1: Làm thế nào tìm được các điểm *tương ứng* giữa hai ảnh khi góc nhìn, ánh sáng, tỉ lệ có thể thay đổi?

Ý tưởng: Phát hiện và Ghép đặc trưng

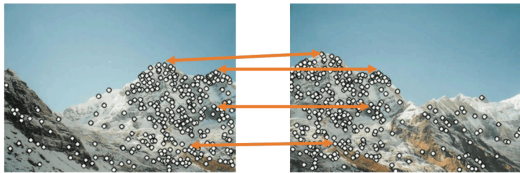


Figure 2: Mỗi điểm: một **keypoint** được phát hiện và ghép nối giữa hai ảnh.

Ba tính chất của một đặc trưng tốt

Repeatability

lặp lại được

Phát hiện tại cùng điểm vật lý dù ảnh thay đổi góc nhìn, tỉ lệ, ánh sáng.

Distinctiveness

phân biệt được

Mô tả vùng lân cận đủ để phân biệt điểm này với mọi điểm khác trong ảnh.

Localization

định vị chính xác

Toạ độ (x, y) xác định chính xác, không bị phân tán trên vùng rộng.

Vùng phẳng: Không có thông tin định vị

Xét cửa sổ w trượt trên ảnh f .

Đo sự thay đổi cường độ khi dịch (u, v) :

$$E(u, v) = \sum_{(x, y) \in w} [f(x+u, y+v) - f(x, y)]^2$$



$E(u, v) \approx 0$ — mọi hướng

Vùng phẳng: $f(x, y) \approx \text{const}$

$$E(u, v) \approx 0 \quad \forall (u, v)$$

Dịch cửa sổ theo bất kỳ hướng nào

$\Rightarrow$ cường độ không đổi

$\Rightarrow$ **không xác định được vị trí.**

Cạnh: Chỉ một hướng có thông tin

Cạnh ngang: f thay đổi mạnh theo y , không đổi theo x .

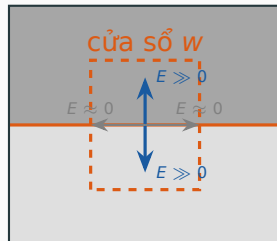
$$E(u, 0) \approx 0 \quad (\text{dọc theo cạnh})$$

$$E(0, v) \gg 0 \quad (\text{vuông góc cạnh})$$

Biết cửa sổ nằm *trên* cạnh, nhưng không biết

vị trí nào dọc theo cạnh — mơ hồ 1 chiều.

⇒ **Không đủ tốt** để định vị 2D.



Góc: Mọi hướng đều có thông tin

Góc: f thay đổi mạnh theo *mọi* hướng.

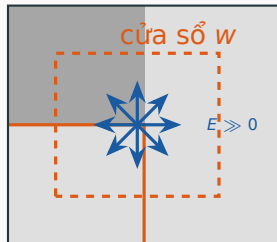
$$E(u, v) \gg 0 \quad \forall (u, v) \neq (0, 0)$$

Dịch cửa sổ theo bất kỳ hướng nào
 $\Rightarrow$ cường độ thay đổi rõ rệt
 $\Rightarrow$ **xác định được vị trí 2D duy nhất.**

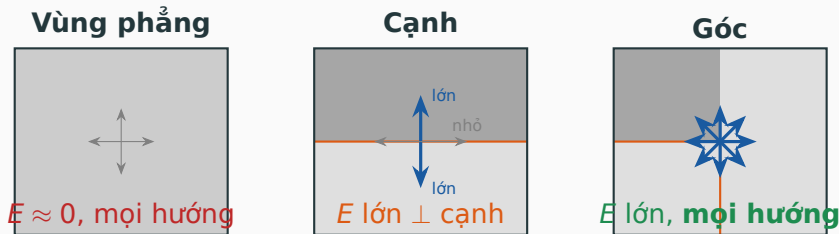
Định nghĩa hoạt động

Một **góc (corner)** là điểm mà

$E(u, v)$ lớn theo **mọi hướng** (u, v) .



Tóm tắt: Ba loại vùng ảnh



Bài toán: tính $E(u, v)$ một cách **hiệu quả** mà không cần dịch cửa sổ thực sự.

Xấp xỉ $E(u, v)$ bằng khai triển Taylor

Khai triển Taylor bậc nhất của $f(x + u, y + v)$:

$$f(x + u, y + v) \approx f(x, y) + u f_x(x, y) + v f_y(x, y)$$

Thế vào $E(u, v)$:

$$E(u, v) \approx \sum_{(x,y) \in w} (u f_x + v f_y)^2$$

Viết dưới dạng ma trận:

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\sum_{(x,y) \in w} \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix}}_M \begin{bmatrix} u \\ v \end{bmatrix} = \mathbf{d}^\top M \mathbf{d}$$

Ma trận M gọi là **structure tensor** (hay second-moment matrix).

Structure Tensor M

$$M = \sum_{(x,y) \in W} \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} = \begin{bmatrix} \sum f_x^2 & \sum f_x f_y \\ \sum f_x f_y & \sum f_y^2 \end{bmatrix}$$

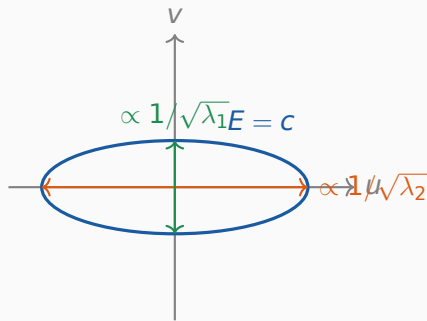
M là ma trận **thực đối xứng** nên luôn có:

hai giá trị riêng thực $\lambda_1 \geq \lambda_2 \geq 0$
và hai vector riêng trực giao.

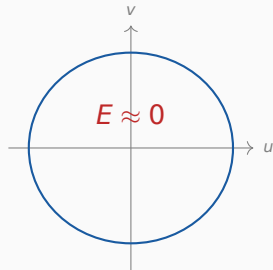
$E(u, v) = \mathbf{d}^\top M \mathbf{d}$ là một **dạng toàn phương**.

Hình học của E trong không gian (u, v)

là một **elipsoid** với bán trục
 $\propto \lambda_1, \lambda_2$.

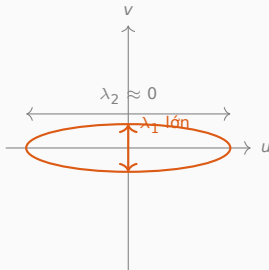


Phân tích Eigenvalue của M



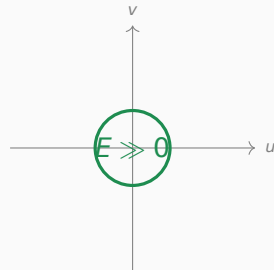
Vùng phẳng

$$\lambda_1 \approx \lambda_2 \approx 0$$



Cạnh

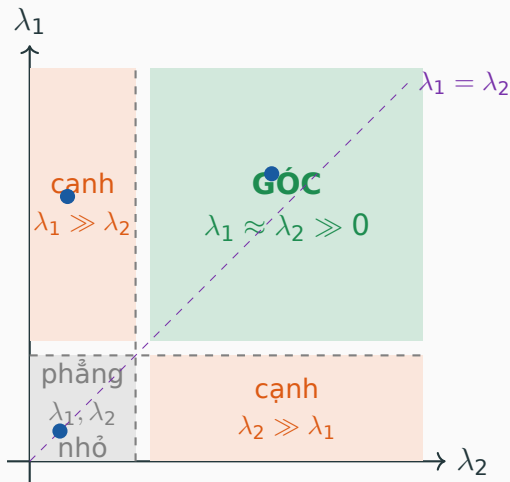
$$\lambda_1 \gg \lambda_2 \approx 0$$



Góc

$$\lambda_1 \approx \lambda_2 \gg 0$$

Không gian Eigenvalue: Phân loại vùng ảnh



Từ M đến Bộ phát hiện góc

Vấn đề thực tế: tính eigenvalue tốn kém — cần phân tích ma trận 2×2 tại *mỗi pixel*.

Quan sát: với ma trận 2×2 :

$$\lambda_1 + \lambda_2 = \text{trace}(M) = f_x^2 + f_y^2$$

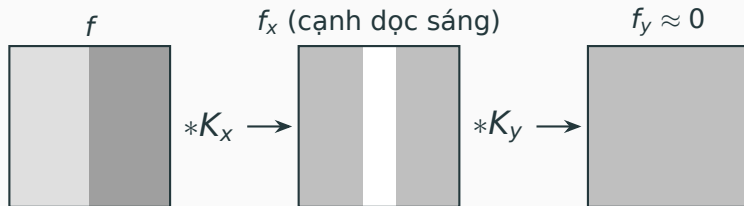
$$\lambda_1 \cdot \lambda_2 = \det(M) = f_x^2 f_y^2 - (f_x f_y)^2$$

$\Rightarrow$ Có thể phân loại vùng ảnh **chỉ từ** $\det(M)$ và $\text{trace}(M)$ mà **không cần tính eigenvalue tường minh**.

Tính Gradient ảnh: Toán tử Sobel

f_x và f_y được tính bằng tích chập với kernel Sobel:

$$K_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad f_x = f * K_x \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \quad f_y = f * K_y$$



Sobel là xấp xỉ đạo hàm riêng

Đạo hàm riêng rời rạc theo x (sai phân trung tâm):

$$\frac{\partial f}{\partial x} \approx \frac{f(x+1, y) - f(x-1, y)}{2} \rightarrow \text{kernel } [-1, 0, +1]$$

Sobel thêm **làm mịn Gaussian theo chiều vuông góc** để giảm nhiễu:

$$K_x = \underbrace{\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}}_{\text{Gaussian}} \underbrace{\begin{bmatrix} -1 & 0 & +1 \end{bmatrix}}_{\text{sai phân}} \Rightarrow \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Tính chất quan trọng

Sobel = tích chập phân ly (separable) $\Rightarrow$ tính hiệu quả:

áp kernel 1×3 lên hàng, rồi kernel 3×1 lên cột.

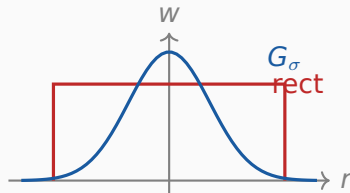
Làm mịn Structure Tensor bằng Gaussian

Thay tổng cứng trong cửa sổ w bằng **trọng số Gaussian**:

$$M = G_{\sigma} * \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} = \begin{bmatrix} G_{\sigma} * f_x^2 & G_{\sigma} * f_x f_y \\ G_{\sigma} * f_x f_y & G_{\sigma} * f_y^2 \end{bmatrix}$$

Tại sao dùng Gaussian thay vì cửa sổ chữ nhật?

- Pixel gần tâm đóng góp *nhiều hơn*
- Giảm hiệu ứng biên cứng của cửa sổ
- Kết quả mượt hơn, ít nhiễu hơn



Harris: Hàm phản hồi R

Harris & Stephens (1988) đề xuất tránh tính eigenvalue bằng:

$$R = \det(M) - k \cdot [\text{trace}(M)]^2$$

Với ma trận 2×2 :

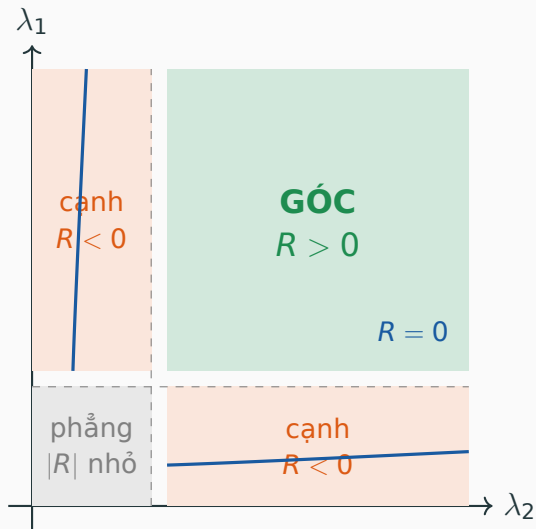
$$\det(M) = \lambda_1 \lambda_2, \quad \text{trace}(M) = \lambda_1 + \lambda_2$$

Thế vào:

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

- Tham số $k \in [0.04, 0.06]$ — thường chọn $k = 0.05$
- R lớn dương $\Rightarrow$ **góc**
- R lớn âm $\Rightarrow$ **cạnh**
- $|R|$ nhỏ $\Rightarrow$ **vùng phẳng**

Hình học của hàm phản hồi R



Tại sao không tính eigenvalue trực tiếp?

Tính eigenvalue của M tại mỗi pixel đòi hỏi:

$$\det(M - \lambda I) = 0 \Rightarrow \lambda^2 - \text{trace}(M)\lambda + \det(M) = 0$$

$$\lambda_{1,2} = \frac{\text{trace}(M) \pm \sqrt{\text{trace}(M)^2 - 4 \det(M)}}{2}$$

Mỗi pixel cần: 1 phép căn bậc hai + vài phép nhân/chia.

Hàm Harris R chỉ cần:

$$R = \underbrace{(f_x^2 f_y^2 - f_{xy}^2)}_{\det(M)} - k \underbrace{(f_x^2 + f_y^2)^2}_{\text{trace}(M)^2}$$

Không cần căn bậc hai — **nhANH hơn đáng kể** khi xử lý toàn bộ ảnh.

Quy trình Harris Corner Detector



Bước 1-3: tính và tổng hợp gradient.

Bước 4: làm mịn mỗi thành phần của M bằng Gaussian σ (thường $\sigma = 1-2$).

Bước 5: tính R tại mỗi pixel.

Bước 6: Non-Maximum Suppression (NMS)

- Giữ lại pixel p nếu $R(p)$ là cực đại trong lân cận 3×3 (hoặc 5×5).
- Loại bỏ các pixel có $R < \text{ngưỡng } T$.

Non-Maximum Suppression (NMS)

Bản đồ R (trước NMS)

0.1	0.3	0.2	0.0
0.5	2.8	1.1	0.4
0.2	1.3	0.6	0.1
0.0	0.2	0.1	0.0

NMS
→

Sau NMS

0	0	0	0
0	●	0	0
0	0	0	0
0	0	0	0

Chỉ giữ lại pixel có R **lớn nhất cục bộ** và $R > T$ (ngưỡng).
Tránh phát hiện nhiều điểm tại cùng một góc.

Tham số k : Ảnh hưởng đến phân loại

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2 = \det(M) - k \cdot \text{trace}(M)^2$$

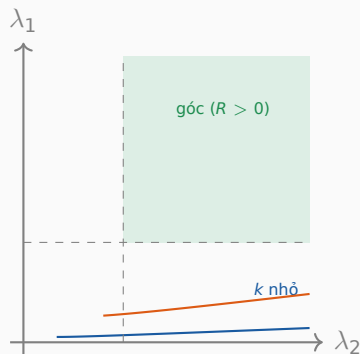
Đường $R = 0$ trong không gian (λ_1, λ_2) :

$$\lambda_1 \lambda_2 = k(\lambda_1 + \lambda_2)^2$$

$$\frac{\lambda_1 \lambda_2}{(\lambda_1 + \lambda_2)^2} = k$$

Vế trái đạt cực đại $\frac{1}{4}$ khi $\lambda_1 = \lambda_2$.

- k nhỏ $\Rightarrow$ đường $R = 0$ ra xa gốc $\Rightarrow$ phát hiện *nhiều* góc hơn
- k lớn $\Rightarrow$ chỉ giữ góc *rõ ràng* hơn



Harris: Bất biến với phép quay

Khi ảnh bị **quay** góc θ , gradient biến đổi theo:

$$\begin{bmatrix} f_{x'} \\ f_{y'} \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} f_x \\ f_y \end{bmatrix}$$

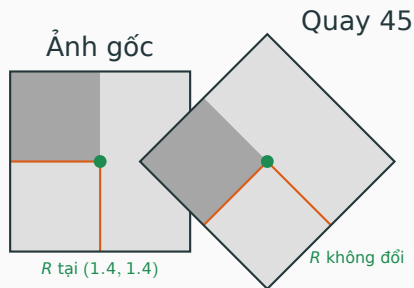
Structure tensor sau quay:

$$M' = R_\theta M R_\theta^\top$$

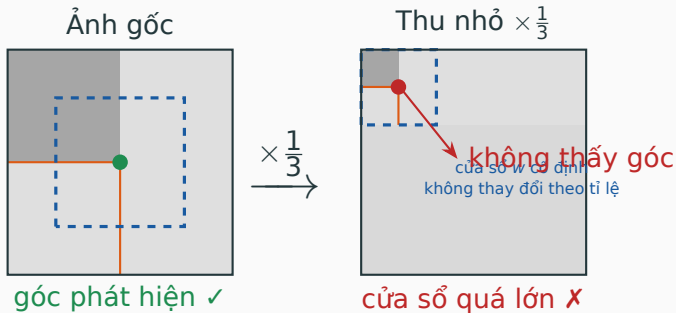
$\det$ và trace **bất biến** với phép biến đổi trực giao:

$$\det(M') = \det(M), \quad \text{trace}(M') = \text{trace}(M)$$

$\Rightarrow R$ không đổi khi ảnh bị quay.



Hạn chế: Harris không bất biến với tỉ lệ



⇒ Cần **phân tích đa thang** để giải quyết — bài sau (SIFT).

Shi-Tomasi (1994): Tiêu chí đơn giản hơn

Shi & Tomasi đề xuất thay hàm R bằng:

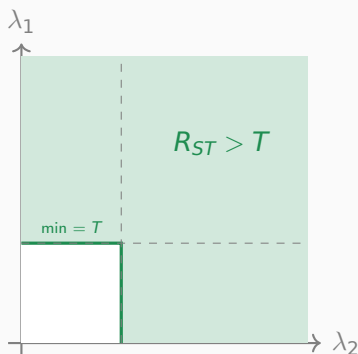
$$R_{ST} = \min(\lambda_1, \lambda_2)$$

Lý luận: một góc tốt là nơi *eigenvalue* nhỏ nhất còn lớn.

Nếu $\min(\lambda_1, \lambda_2) > T$ thì điểm đó là góc.

So sánh với Harris:

- Harris: $R = \det - k \cdot \text{trace}^2$
 - không cần tính eigenvalue
 - nhưng phụ thuộc tham số k
- Shi-Tomasi: $R = \min(\lambda_1, \lambda_2)$
 - cần tính eigenvalue
 - không có tham số k



So sánh: Harris vs Shi-Tomasi

	Harris (1988)	Shi-Tomasi (1994)
Hàm phản hồi	$\det(M) - k \cdot \text{trace}(M)^2$	$\min(\lambda_1, \lambda_2)$
Tính eigenvalue	Không (nhẹ hơn)	Có (chậm hơn)
Tham số	$k \in [0.04, 0.06]$	Không có
Bất biến phép quay	Có	Có
Bất biến tỉ lệ	Không	Không
Dùng trong thực tế	<code>cv2.cornerHarris</code>	<code>cv2.goodFeaturesToTrack</code>

★ Ý tưởng cốt lõi

Harris tránh tính eigenvalue bằng cách dùng

$$R = \det(M) - k \cdot \text{trace}(M)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

$R > 0$: góc $R < 0$: cạnh $|R| \approx 0$: vùng phẳng

Shi-Tomasi đơn giản hoá thành $\min(\lambda_1, \lambda_2) > T$
— không cần tham số k , kết quả ổn định hơn.

Demo: Harris và Shi-Tomasi trong Python

Những gì sẽ thực hiện trong notebook:

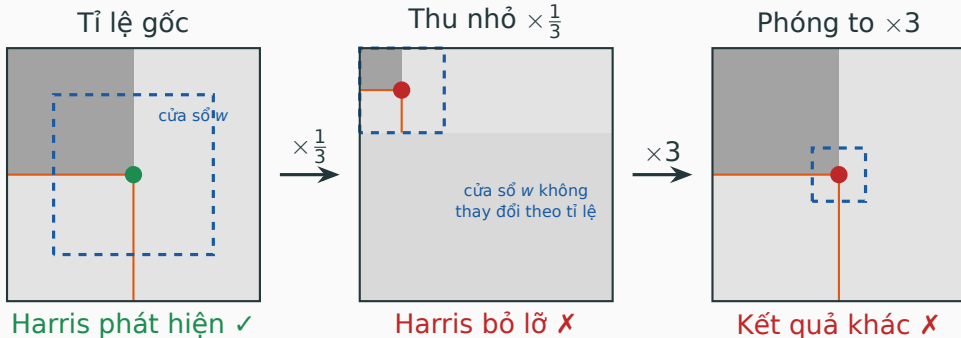
1. Xây dựng ảnh tổng hợp có vùng phẳng, cạnh, và góc bằng NumPy
2. Tính f_x, f_y bằng Sobel; tính $f_x^2, f_x f_y, f_y^2$
3. Làm mịn bằng Gaussian; tính $R = \det(M) - k \cdot \text{trace}(M)^2$
4. Hiển thị bản đồ R dưới dạng heatmap — quan sát các vùng
5. Áp NMS và ngưỡng; vẽ các góc phát hiện lên ảnh gốc
6. So sánh với `cv2.goodFeaturesToTrack` (Shi-Tomasi)

Không gian Thang đo & Kim tự tháp Gaussian

Tuần 3 — Phát hiện đặc trưng bất biến thang đo

ĐH GTVT TP.HCM · Môn 121036

Vấn đề: Harris thất bại khi tỉ lệ thay đổi



Yêu cầu: Bất biến thang đo (Scale Invariance)

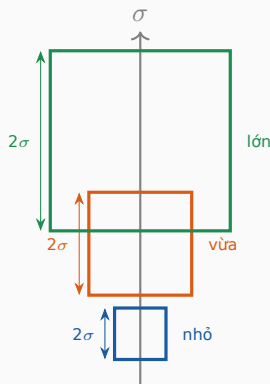
Mục tiêu: phát hiện cùng điểm ảnh dù ảnh bị phóng to hay thu nhỏ bất kỳ.

Ý tưởng cốt lõi:

Thay vì dùng *một* cửa sổ kích thước cố định, hãy phân tích ảnh ở **nhiều thang đo** cùng lúc.

Câu hỏi hình thức:

Làm thế nào xây dựng một *họ biểu diễn* $L(x, y, \sigma)$ của ảnh f tại mọi thang đo σ ?



Định nghĩa: Không gian Thang đo (Scale Space)

Định nghĩa (Witkin 1983, Koenderink 1984, Lindeberg 1994)

Không gian thang đo tuyến tính của ảnh $f(x, y)$ là họ hàm:

$$L(x, y, \sigma) = G_{\sigma}(x, y) * f(x, y)$$

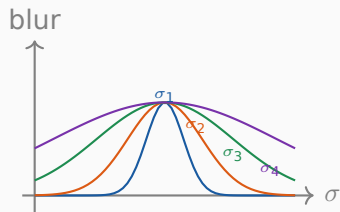
với G_{σ} là kernel Gaussian 2D:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Tại sao Gaussian?

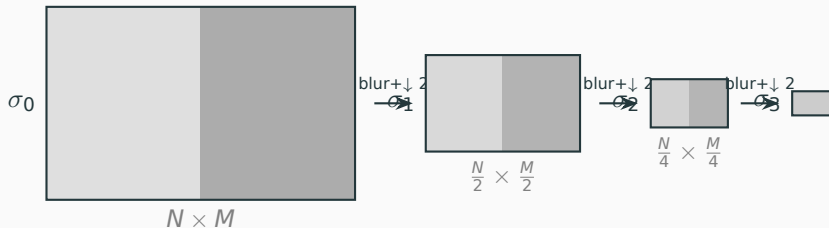
Lindeberg (1994) chứng minh: Gaussian là *duy nhất* thoả mãn đồng thời:

- Tuyến tính và dịch chuyển bất biến
- Không tạo thêm cực trị khi σ tăng
- Đối xứng, tách biệt theo x và y



Kim tự tháp Gaussian

Trong thực tế, scale space liên tục được **rời rạc hoá** thành kim tự tháp:



Mỗi tầng: **blur** với G_σ rồi **hạ mẫu 2** (downsample).

Sigma tăng theo cấp số nhân: $\sigma_k = \sigma_0 \cdot 2^k$.

Tính chất nửa nhóm của Gaussian

Định lý

$$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2 + \sigma_2^2}}$$

Hệ quả quan trọng:

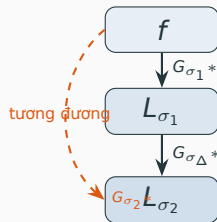
Để đi từ thang σ_1 lên thang $\sigma_2 > \sigma_1$, chỉ cần blur thêm một lần với σ_Δ :

$$L(x, y, \sigma_2) = G_{\sigma_\Delta} * L(x, y, \sigma_1), \quad \sigma_\Delta = \sqrt{\sigma_2^2 - \sigma_1^2}$$

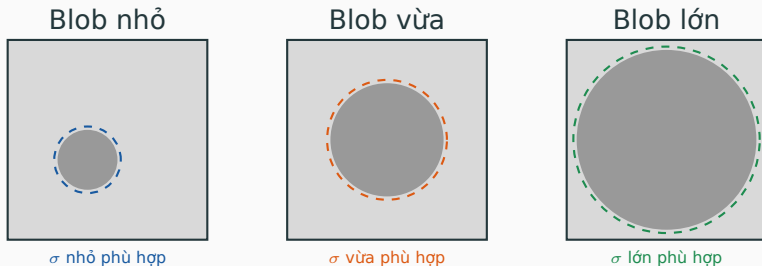
Ý nghĩa thực tế:

Xây dựng kim tự tháp bằng cách blur *tăng dần* thay vì blur từ ảnh gốc tại mỗi tầng.

Tránh tích chập kernel lớn nhiều lần.



σ và kích thước cấu trúc bắt được



Mỗi cấu trúc có một **thang đặc trưng** σ^* mà tại đó phản hồi của bộ lọc đạt **cực đại**.

Nhiệm vụ: tìm cả *vị trí* (x, y) lẫn *thang đo* σ^* .

Laplacian of Gaussian (LoG)

Để phát hiện blob tại thang σ , dùng **Laplacian of Gaussian**:

$$\nabla^2 G_\sigma(x, y) = \frac{\partial^2 G_\sigma}{\partial x^2} + \frac{\partial^2 G_\sigma}{\partial y^2} = \frac{x^2 + y^2 - 2\sigma^2}{2\pi\sigma^6} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

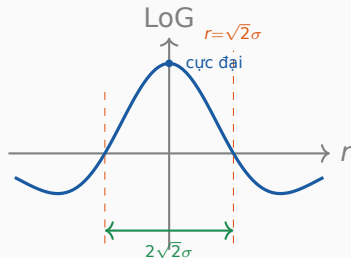
Phản hồi lớn khi blob có bán kính
 $r \approx \sqrt{2}\sigma$.

Chuẩn hoá thang đo:

Nhân thêm σ^2 để so sánh phản hồi
qua các thang:

$$\text{LoG}_{\text{norm}} = \sigma^2 \nabla^2 G_\sigma * f$$

Không có σ^2 : phản hồi giảm khi σ
tăng, làm mất cực đại thật ở thang
lớn.

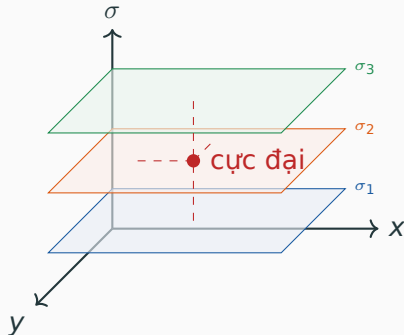


Phát hiện Blob: Tìm cực trị trong (x, y, σ)

Áp LoG_{norm} tại nhiều thang đo. Một blob được phát hiện khi:

$\sigma^2 \nabla^2 G_\sigma * f$ đạt cực đại (hoặc cực tiểu) cục bộ

trong **cả ba chiều**: x , y , và σ .



LoG tốn kém — Xấp xỉ bằng DoG

Vấn đề: tích chập với $\nabla^2 G_\sigma$ tốn kém, đặc biệt khi cần tính tại nhiều thang đo.

Xấp xỉ hiệu quả: Difference of Gaussians (DoG).

Từ phương trình nhiệt (heat equation):

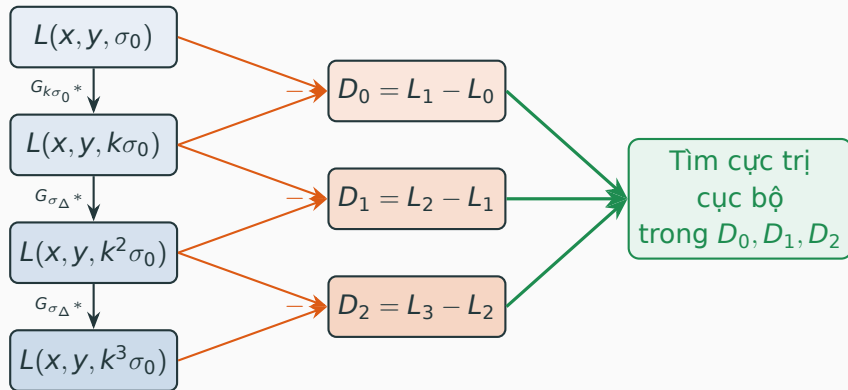
$$\frac{\partial G_\sigma}{\partial \sigma} = \sigma \nabla^2 G_\sigma \quad \Rightarrow \quad \sigma \nabla^2 G_\sigma \approx \frac{G_{k\sigma} - G_\sigma}{k - 1}$$

Difference of Gaussians

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) = (G_{k\sigma} - G_\sigma) * f \approx (k - 1)\sigma^2 \nabla^2 G_\sigma * f$$

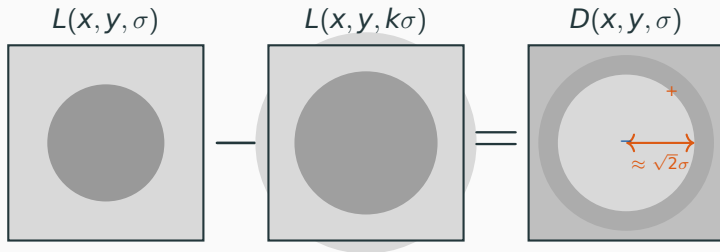
Với k nhỏ (ví dụ $k = 2^{1/s}$, s là số tầng trong octave), xấp xỉ này rất chính xác.

DoG: Tại sao rẻ hơn LoG?



DoG = **phép trừ** hai ảnh Gaussian liên tiếp đã tính sẵn.
Không cần tích chập thêm — **gần như miễn phí**.

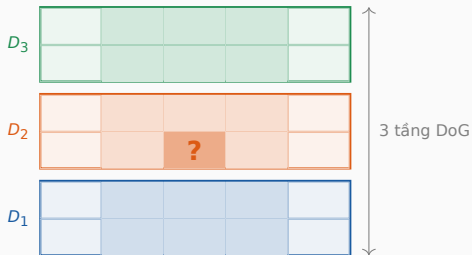
DoG: Minh họa trực quan



DoG cho phản hồi cực đại khi blob có bán kính $r \approx \sqrt{2}\sigma$, giống hệt LoG nhưng tính bằng phép trừ đơn giản.

Tìm cực trị cục bộ trong không gian DoG

Một điểm (x_0, y_0, σ_0) là ứng viên keypoint nếu $D(x_0, y_0, \sigma_0)$ là cực trị so với **26 lân cận** (9 trong cùng thang + 9 trên + 8 dưới):



26 lân cận được so sánh

Octave: Cấu trúc kim tự tháp cho SIFT

Một **octave** = tập ảnh Gaussian tại **cùng độ phân giải**,

sigma tăng từ σ đến 2σ .

Trong mỗi octave có $s + 3$ ảnh Gaussian

$\Rightarrow s + 2$ ảnh DoG

$\Rightarrow s$ tầng hữu dụng để tìm cực trị.

Sang octave tiếp theo:

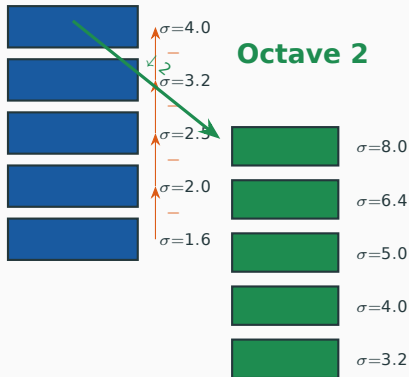
hạ mẫu 2 từ ảnh Gaussian giữa octave.

$$\sigma_{\text{base, octave}+1} = 2\sigma_{\text{base, octave}}$$

Thông số chuẩn (Lowe 2004):

$$s = 3, \sigma_0 = 1.6, k = 2^{1/s}.$$

Octave 1



So sánh: LoG và DoG

	LoG	DoG
Công thức	$\sigma^2(G_{xx} + G_{yy}) * f$	$L(k\sigma) - L(\sigma)$
Cách tính	Tích chập trực tiếp	Phép trừ hai ảnh
Chi phí tính toán	Cao (kernel lớn)	Thấp (tận dụng ảnh Gaussian có sẵn)
Chất lượng xấp xỉ	Chính xác	Xấp xỉ tốt với k nhỏ
Dùng trong thực tế	Ít (nghiên cứu)	SIFT, ORB

DoG là lựa chọn tiêu chuẩn trong thực tế nhờ tái sử dụng ảnh Gaussian đã tính trong kim tự tháp.

★ Ý tưởng cốt lõi

Scale space $L(x, y, \sigma) = G_\sigma * f$ biểu diễn ảnh ở mọi thang đo đồng thời.

DoG xấp xỉ LoG bằng phép trừ đơn giản:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 G_\sigma * f$$

Keypoint là cực trị cục bộ của D trong cả ba chiều (x, y, σ) .

Tọa độ σ^* cho biết **thang đặc trưng**
— kích thước tự nhiên của blob đó.

Demo: Kim tự tháp Gaussian và DoG trong Python

Những gì sẽ thực hiện trong notebook:

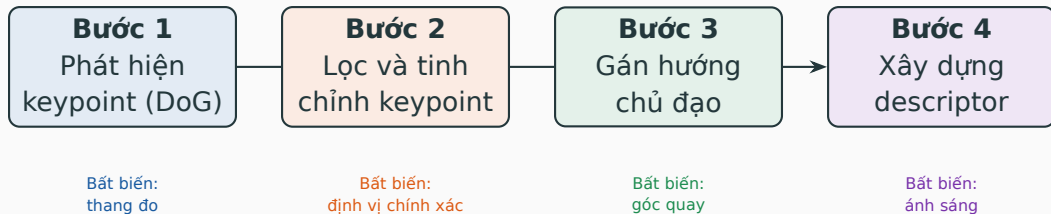
1. Xây dựng kim tự tháp Gaussian 3 octave ($s = 3$) bằng `scipy.ndimage.gaussian_filter`
2. Tính các ảnh DoG từ hiệu ảnh Gaussian liên tiếp; hiển thị lưới DoG
3. Kiểm tra tính chất nửa nhóm: so sánh $G_{\sigma_1} * (G_{\sigma_2} * f)$ với $G_{\sqrt{\sigma_1^2 + \sigma_2^2}} * f$
4. Tìm cực trị cục bộ trong 26 lân cận; đánh dấu keypoint trên ảnh gốc
5. So sánh keypoint ở tỉ lệ gốc và thu nhỏ $\times \frac{1}{2}$:
Harris: keypoint thay đổi; DoG: keypoint ổn định hơn

SIFT: Scale-Invariant Feature Transform

Tuần 6 — Đặc trưng bất biến thang đo và góc quay

ĐH GTVT TP.HCM · Môn 121036 · Lowe (2004)

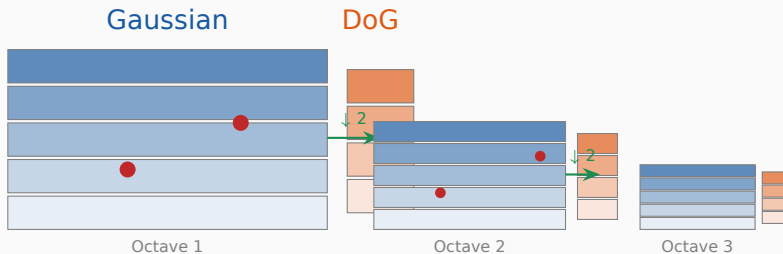
SIFT: Bốn bước, bốn bài toán bất biến



Mỗi bước giải quyết **đúng một** bài toán bất biến.
Đây là thiết kế quan trọng nhất để hiểu SIFT.

Bước 1: Phát hiện Keypoint bằng DoG

Từ bài trước: xây dựng kim tự tháp DoG và tìm cực trị cục bộ.

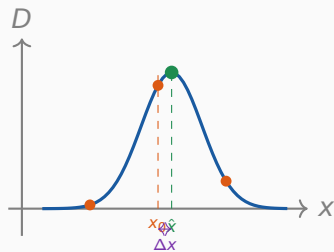


Ứng viên keypoint = cực trị trong 26 lân cận (x, y, σ) .
Bước 1 cho *nhiều* ứng viên — cần lọc tiếp.

Bước 2a: Tinh chỉnh vị trí keypoint

Cực trị của DoG được tìm trên lưới rời rạc.
Tinh chỉnh bằng khai triển Taylor bậc 2 của $D(x, y, \sigma)$:

$$D(\mathbf{x}) \approx D(\mathbf{x}_0) + \left(\frac{\partial D}{\partial \mathbf{x}} \right)^\top \Delta \mathbf{x} + \frac{1}{2} \Delta \mathbf{x}^\top \frac{\partial^2 D}{\partial \mathbf{x}^2} \Delta \mathbf{x}$$



Đạo hàm theo $\Delta \mathbf{x}$ và đặt bằng 0: $\hat{\mathbf{x}} = - \left(\frac{\partial^2 D}{\partial \mathbf{x}^2} \right)^{-1} \frac{\partial D}{\partial \mathbf{x}}$

- $\hat{\mathbf{x}} = (\Delta x, \Delta y, \Delta \sigma)$ là độ lệch khỏi lưới
- Nếu $|\hat{x}|$ hoặc $|\hat{y}| > 0.5$: chuyển sang pixel lân cận và lặp lại
- Giá trị DoG tại vị trí tinh chỉnh:

$$D(\hat{\mathbf{x}}) = D(\mathbf{x}_0) + \frac{1}{2} \left(\frac{\partial D}{\partial \mathbf{x}} \right)^\top \hat{\mathbf{x}}$$

Bước 2b: Loại bỏ keypoint yếu và keypoint trên cạnh

Loại 1 — Độ tương phản thấp:

Nếu $|D(\hat{\mathbf{x}})| < 0.03$: loại bỏ (không ổn định với nhiễu).

Loại 2 — Keypoint trên cạnh (edge keypoint):

Cạnh có một eigenvalue của M lớn, một nhỏ.

Dùng tỉ số cong (ratio of curvatures) từ Hessian của D :

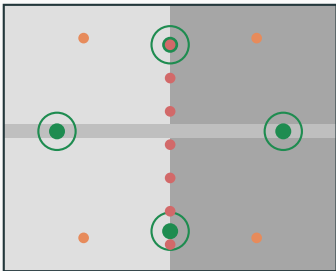
$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$
$$\frac{\text{trace}(H)^2}{\det(H)} = \frac{(\lambda_1 + \lambda_2)^2}{\lambda_1 \lambda_2} = \frac{(r + 1)^2}{r}, \quad r = \frac{\lambda_1}{\lambda_2}$$

Điều kiện giữ lại

$$\frac{\text{trace}(H)^2}{\det(H)} < \frac{(r_{\text{th}} + 1)^2}{r_{\text{th}}}, \quad r_{\text{th}} = 10 \text{ (Lowe 2004)}$$

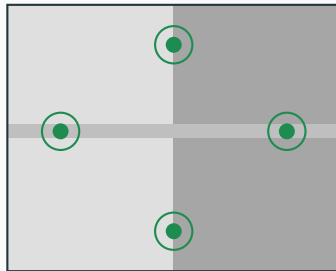
Bước 2b: Minh hoạ — Trước và sau khi lọc

Ứng viên thô (sau Bước 1)



lọc
→

Sau khi lọc (Bước 2)



● giữ lại ● cạnh (loại) tương phản thấp (loại)

Bước 3: Gán Hướng Chủ đạo

Mục tiêu: gán một **hướng tham chiếu** cho mỗi keypoint để descriptor sau đó được tính *tương đối* với hướng đó.

Quy trình:

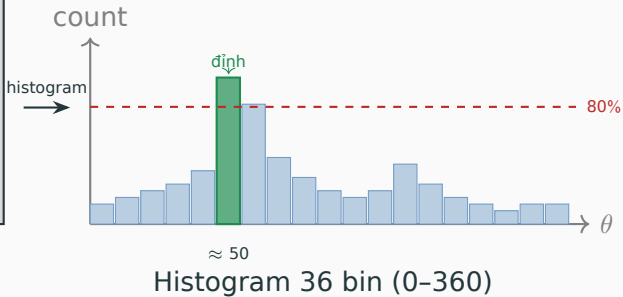
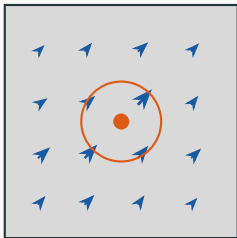
1. Tại thang σ của keypoint, tính gradient mỗi pixel trong vùng lân cận:

$$m(x, y) = \sqrt{(L_x)^2 + (L_y)^2}, \quad \theta(x, y) = \arctan \frac{L_y}{L_x}$$

2. Xây dựng **histogram hướng** 36 bin (10 mỗi bin),
trọng số = biên độ gradient $\times$ Gaussian tâm tại keypoint
3. Hướng chủ đạo = bin cao nhất (đỉnh histogram)
4. Nếu có bin khác $\geq 80\%$ đỉnh: tạo **keypoint mới** với hướng đó

Bước 3: Histogram Hướng Gradient

Gradient lân cận



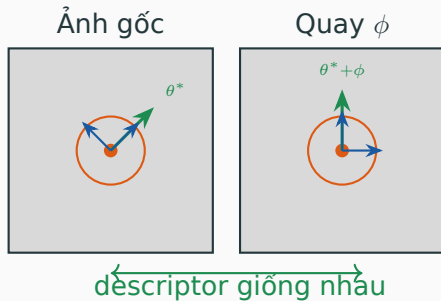
Bước 3: Tại sao gán hướng đảm bảo bất biến góc quay?

Sau khi gán hướng θ^* , toàn bộ descriptor được tính trong **hệ tọa độ cục bộ** xoay theo θ^* .

Khi ảnh bị quay ϕ :

- Hướng gradient mỗi pixel tăng thêm ϕ
- Đỉnh histogram dịch sang $\theta^* + \phi$
- Hệ tọa độ descriptor cũng xoay ϕ
- **Descriptor không đổi**

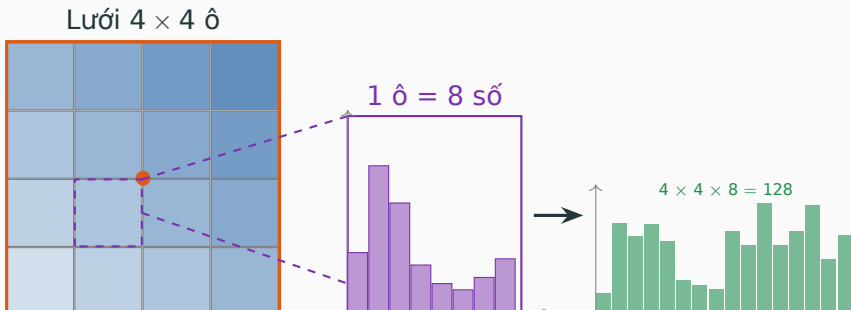
Tương tự: descriptor được tính tại thang σ^*



Bước 4: Xây dựng Descriptor — Lưới 4×4

Tại mỗi keypoint (đã biết σ^* và θ^*):

1. Lấy vùng 16×16 pixel xung quanh (trong hệ toạ độ cục bộ xoay θ^*)
2. Chia thành lưới 4×4 ô con, mỗi ô 4×4 pixel
3. Trong mỗi ô: xây dựng histogram hướng **8 bin**
4. Nối 16 histogram lại: $4 \times 4 \times 8 = \mathbf{128}$ chiều



Bước 4: Chuẩn hoá Descriptor

Vector thô $\mathbf{d} \in \mathbb{R}^{128}$ nhạy với thay đổi ánh sáng.

Lowe (2004) đề xuất chuẩn hoá hai bước:

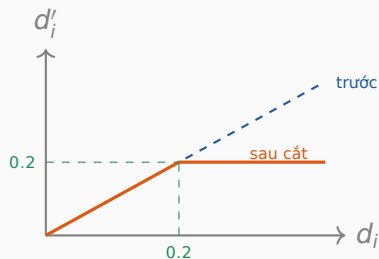
Bước 4a — Chuẩn hoá L2:

$$\mathbf{d} \leftarrow \frac{\mathbf{d}}{\|\mathbf{d}\|_2}$$

Loại bỏ ảnh hưởng của **nhân độ sáng** (tất cả gradient nhân cùng hằng số).

Bước 4b — Cắt ngưỡng 0.2:

$$d_i \leftarrow \min(d_i, 0.2)$$



Descriptor cuối cùng: 128 số thực trong $[0, 1]$.

Thường lượng tử hoá thành **số nguyên 8-bit** để tiết kiệm bộ

Ghép Keypoint: Khoảng cách Euclid và Ratio Test

Cách ghép cơ bản:

Với mỗi keypoint $\mathbf{d}_i$ trong ảnh A, tìm keypoint gần nhất trong ảnh B:

$$j^* = \arg \min_j \|\mathbf{d}_i - \mathbf{d}_j\|_2$$

Vấn đề: khoảng cách tuyệt đối không có ý nghĩa riêng lẻ.

Một điểm xấu có thể vẫn là “gần nhất” nếu không có ứng viên tốt.

Ratio Test (Lowe 2004)

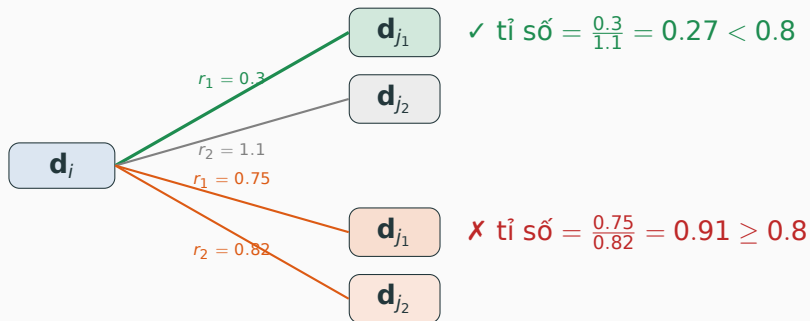
$$\frac{\|\mathbf{d}_i - \mathbf{d}_{j_1^*}\|_2}{\|\mathbf{d}_i - \mathbf{d}_{j_2^*}\|_2} < 0.8$$

Chấp nhận ghép chỉ khi khoảng cách đến *gần nhất* nhỏ hơn 80% khoảng cách đến *gần nhì*.

Nếu tỉ số ≥ 0.8 : không phân biệt được $\rightarrow$ bỏ cặp ghép đó.

Ratio Test: Minh họa

Trường hợp 1: ghép tốt



Tóm tắt: SIFT bất biến với những gì?

Biến đổi	Cơ chế	Bất biến?
Tịnh tiến	DoG phát hiện theo vị trí (x, y)	Có
Thang đo	Cực trị theo σ , descriptor tại σ^*	Có
Góc quay	Hệ toạ độ cục bộ theo θ^*	Có
Độ sáng cộng	Gradient loại bỏ offset ánh sáng	Có
Độ sáng nhân	Chuẩn hoá L2	Có
Phép chiếu phối cảnh nhỏ	Mô tả cục bộ (bố cục 4×4)	Gần đúng 100
Biến đổi phối cảnh lớn	(Cần ASIFT hoặc phương pháp khác)	Không

Độ phức tạp tính toán SIFT

Xây dựng kim tự tháp Gaussian:

$O(MN \log MN)$ với FFT, hoặc $O(MN)$ với kernel tách biệt.

Tính DoG: $O(MN)$ mỗi tầng.

Descriptor: $O(K \cdot 128)$ với K keypoint.

Ghép (brute-force): $O(K_A \cdot K_B \cdot 128)$.

Thực tế dùng **kd-tree** hoặc **FLANN**:
 $O(K \log K)$ trung bình.

Hạn chế của SIFT

- Chậm với ảnh lớn hoặc video real-time
- Descriptor 128D tốn bộ nhớ khi K lớn
- Bằng sáng chế (hết hạn 2020):
nay dùng tự do trong OpenCV
- Không bất biến với biến đổi phối cảnh lớn

★ Ý tưởng cốt lõi

SIFT giải quyết bốn bài toán bất biến bằng bốn bước độc lập:

Bước 1 DoG $\Rightarrow$ bất biến **thang đo**

Bước 2 Taylor + Hessian $\Rightarrow$ định vị **chính xác**

Bước 3 Histogram hướng $\Rightarrow$ bất biến **góc quay**

Bước 4 Chuẩn hoá L2 $\Rightarrow$ bất biến **ánh sáng**

Descriptor 128D = 4×4 ô $\times$ 8 bin hướng gradient.

ORB: Oriented FAST and Rotated BRIEF

Tuần 3 — Đặc trưng nhị phân nhanh

ĐH GTVT TP.HCM · Môn 121036

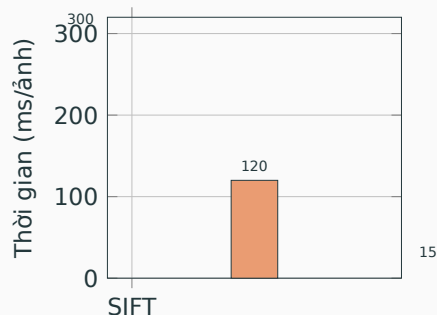
Động lực: Tại sao cần ORB?

SIFT mạnh, nhưng:

- Descriptor 128D số thực $\Rightarrow$ **tốn bộ nhớ**
- Ghép bằng khoảng cách Euclid $\Rightarrow$ **chậm**
- Kim tự tháp Gaussian + nhiều tích chập $\Rightarrow$ **không real-time**

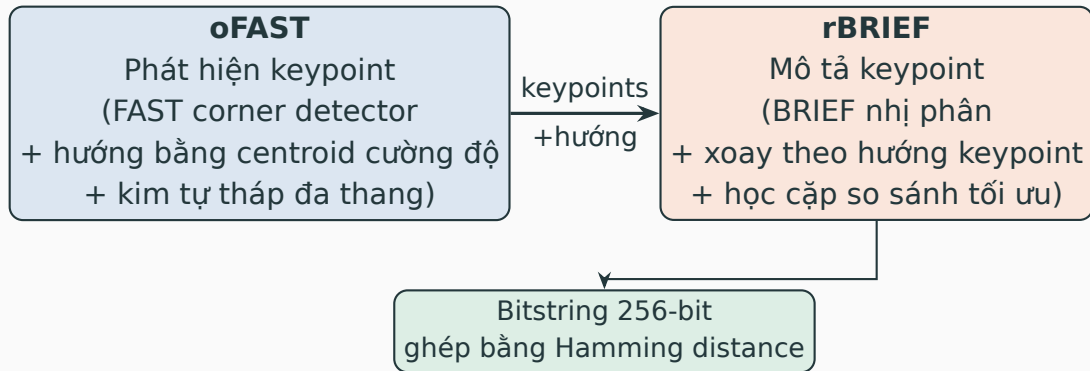
Mục tiêu ORB :

- Nhanh hơn SIFT **100 lần**
- Descriptor **nhị phân** (bitstring)
— ghép bằng XOR



Thời gian xử lý ảnh 640×480

Kiến trúc ORB: Hai thành phần



FAST: Features from Accelerated Segment Test

Rosten & Drummond (2006). Ý tưởng: pixel p là góc nếu tồn tại n **pixel liên tiếp** trên vòng tròn Bresenham bán kính 3 đều *sáng hơn hoặc tối hơn* p một ngưỡng t . **Vòng tròn Bresenham bán kính 3: 16 pixel.**

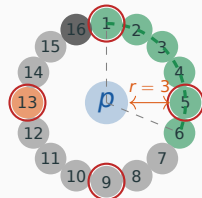
Điều kiện góc (với $n = 12$):

$\exists$ cung 12 pixel liên tiếp sao cho

$$I_p^{\text{pixel}} > I_p + t \quad (\text{tất cả sáng hơn})$$

hoặc

$$I_p^{\text{pixel}} < I_p - t \quad (\text{tất cả tối hơn})$$



cung sáng hơn (6/16 pixel)

□ 4 pixel kiểm tra nhanh

Kiểm tra nhanh

(loại bỏ sớm): chỉ kiểm tra

4 pixel tại vị trí 1, 5, 9, 13 trước.

Nếu ≥ 3 pixel không thoả điều kiện: bỏ

FAST: NMS và Điểm số

FAST không tự có điểm số — ORB thêm **Harris response** làm điểm:

$$s_p = R_{\text{Harris}}(p) = \det(M_p) - k \cdot \text{trace}(M_p)^2$$

Tại sao dùng Harris score thay vì score FAST?

- FAST score (số pixel thoả điều kiện) không phân biệt tốt các góc
- Harris score ổn định hơn, NMS hiệu quả hơn

Đa thang đo trong ORB:

Xây dựng **kim tự tháp ảnh** với n tầng, tỉ lệ thu nhỏ s (thường $s = 1.2$):

$$I_k = \text{resize}(I, s^{-k}), \quad k = 0, 1, \dots, n-1$$

Áp FAST trên mỗi tầng, chiếu keypoint về ảnh gốc.

⇒ Bất biến **gần đúng** với thang đo (đơn giản hơn DoG của SIFT).

oFAST: Gán hướng bằng Intensity Centroid

FAST không có hướng. ORB thêm bằng **moment cường độ**.

Định nghĩa moment bậc (p, q) trong patch bán kính r :

$$m_{pq} = \sum_{x,y \in \text{patch}} x^p y^q I(x, y)$$

Centroid cường độ (intensity centroid):

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right)$$

Hướng keypoint = góc từ tâm patch đến centroid:

$$\theta = \arctan \frac{m_{01}}{m_{10}}$$

Tại sao không dùng histogram hướng như SIFT?

Histogram 36 bin đòi hỏi tính gradient tại nhiều pixel và tính histogram.

Intensity centroid chỉ cần **hai tổng đơn giản** m_{10} và m_{01} — nhanh hơn nhiều.

oFAST: Minh hoạ Intensity Centroid

Patch cường độ

80	85	90	85	80
70	90	95	88	72
60	75	78	70	62

$$m_{00} = \sum I(x, y)$$

$$m_{10} = \sum x \cdot I(x, y)$$

$$m_{01} = \sum y \cdot I(x, y)$$

$$C_x = \frac{m_{10}}{m_{00}}$$

$$C_y = \frac{m_{01}}{m_{00}}$$

BRIEF: Binary Robust Independent Elementary Features

Calonder et al. (2010). Ý tưởng: thay vì histogram gradient (128 số thực),
dùng **chuỗi bit** từ các phép so sánh cường độ.

Định nghĩa một **phép kiểm tra nhị phân** tại cặp điểm (p, q) :

$$\tau(P; p, q) = \begin{cases} 1 & \text{nếu } I(p) < I(q) \\ 0 & \text{nếu } I(p) \geq I(q) \end{cases}$$

Descriptor BRIEF = vector nhị phân từ n_d phép kiểm tra:

$$f_{n_d}(P) = \sum_{1 \leq i \leq n_d} 2^{i-1} \tau(P; p_i, q_i) \in \{0, 1\}^{n_d}$$

Chọn cặp (p_i, q_i) như thế nào?

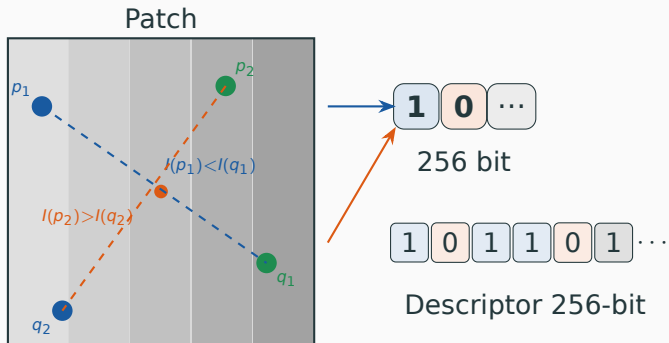
BRIEF dùng phân phối ngẫu nhiên Gaussian.

ORB học cặp tối ưu từ dữ liệu (xem slide sau).

Ưu điểm so sánh với SIFT:

- Lưu trữ: 256 bit thay vì 128 số float (512 byte)
- Ghép: XOR + popcount thay vì khoảng cách Euclid
- Tính toán: nhanh hơn đáng kể

BRIEF: Minh hoạ Cặp So sánh



rBRIEF: Học cặp so sánh tối ưu

Vấn đề của BRIEF ngẫu nhiên:

Các cặp ngẫu nhiên có thể tương quan cao $\Rightarrow$ descriptor kém phân biệt.

ORB dùng phương pháp học (greedy search):

Trên tập huấn luyện lớn, với mỗi keypoint đã biết, tìm tập $n_d = 256$ cặp (p_i, q_i) sao cho:

$$\max \sum_i H(\tau_i) \quad \text{s.t.} \quad |\text{Corr}(\tau_i, \tau_j)| < \epsilon \quad \forall i \neq j$$

- $H(\tau_i)$: entropy của bit thứ i — muốn **gần 0.5** (không thiên vị)
- Ràng buộc tương quan thấp: các bit **gần độc lập** nhau

Kết quả

Tập 256 cặp được **học trước và cố định** trong thư viện ORB.

Không cần học lại khi chạy — chi phí sử dụng bằng BRIEF thông thường.

rBRIEF: Xoay cặp theo hướng keypoint

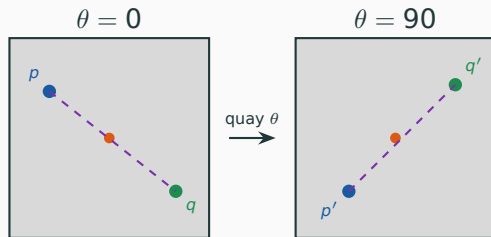
BRIEF không bất biến góc quay. ORB khắc phục bằng cách **xoay các cặp** (p_i, q_i) theo hướng θ của keypoint:

$$\begin{bmatrix} p'_i \\ q'_i \end{bmatrix} = R_\theta \begin{bmatrix} p_i \\ q_i \end{bmatrix}, \quad R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Thực tế: tính trước ma trận cặp đã xoay
cho 30 góc θ (bước 12) và lưu bảng tra.

Khi chạy: tra bảng theo θ của keypoint

$\Rightarrow$ không cần tính sin/cos tại runtime.



Ghép Descriptor: Khoảng cách Hamming

Khoảng cách Hamming giữa hai bitstring $\mathbf{a}, \mathbf{b} \in \{0, 1\}^n$:

$$d_H(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n a_i \oplus b_i = \text{popcount}(\mathbf{a} \oplus \mathbf{b})$$

= số bit khác nhau giữa $\mathbf{a}$ và $\mathbf{b}$.

a:	1	0	1	1	0	1	0	1	...	(256 bit)
b:	1	1	1	0	0	1	1	1		
$\mathbf{a} \oplus \mathbf{b}$:	0	1	0	1	0	0	1	0		

$d_H = 3$ (3 bit khác nhau)

Tại sao nhanh hơn khoảng cách Euclid?

CPU hiện đại có lệnh POPCNT — đếm số bit 1 trong một số nguyên 64-bit trong **1 chu kỳ clock**. Với 256 bit: chỉ cần 4 lệnh POPCNT.

So sánh Tốc độ và Bộ nhớ: ORB vs SIFT

	SIFT	ORB
Phát hiện keypoint	DoG (nhiều tích chập)	FAST ($O(1)$ / pixel)
Bất biến thang đo	Kim tự tháp DoG	Kim tự tháp ảnh
Bất biến góc quay	Histogram 36 bin	Intensity centroid
Descriptor	$128 \times \text{float32} = 512 \text{ byte}$	$256 \text{ bit} = 32 \text{ byte}$
Khoảng cách ghép	Euclid ($O(128)$ phép nhân)	Hamming ($O(4)$ lệnh XOR)
Bộ nhớ (1000 keypoint)	$\approx 512 \text{ KB}$	$\approx 32 \text{ KB}$
Tốc độ tổng thể	baseline	$\approx 100\times$ nhanh hơn

Khi nào dùng ORB? Khi nào dùng SIFT?

Dùng ORB khi:

- Ứng dụng **real-time** (video, robot, AR)
- Tài nguyên hạn chế (thiết bị nhúng, mobile)
- Biến đổi ảnh **nhỏ** (không xoay nhiều, không thay đổi tỉ lệ lớn)
- Cần **miễn phí, mã nguồn mở** hoàn toàn

Dùng SIFT khi:

- Yêu cầu **độ chính xác cao** (stitching, SfM, SLAM)
- Biến đổi ảnh **lớn** (thay đổi góc nhìn, tỉ lệ nhiều)
- Thời gian xử lý không phải ưu tiên
- Ảnh có nhiều **texture phức tạp**

Tốc độ cần thiết



★ Ý tưởng cốt lõi

ORB thay thế từng thành phần
đắt của SIFT bằng xấp xỉ nhanh:

DoG $\rightarrow$ FAST (kiểm tra vòng tròn, $O(1)/\text{pixel}$)

Histogram 36 bin $\rightarrow$ Intensity centroid (m_{01}/m_{10})

$128 \times \text{float32}$ $\rightarrow$ 256 bit (rBRIEF)

Khoảng cách Euclid $\rightarrow$ Hamming = XOR + POPCNT

Kết quả: nhanh hơn $\sim 100\times$, bộ nhớ nhỏ hơn $16\times$,
đủ tốt cho phần lớn ứng dụng thực tế.

Demo: ORB trong Python (OpenCV)

Những gì sẽ thực hiện trong notebook:

1. Tạo hai phiên bản ảnh: gốc và quay 30, thu nhỏ $0.8\times$
2. Phát hiện keypoint và tính descriptor:
`orb = cv2.ORB_create(nfeatures=500)`
`kp, des = orb.detectAndCompute(img, None)`
3. Ghép bằng Hamming distance:
`bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)`
`matches = bf.match(des1, des2)`
4. Sắp xếp matches theo khoảng cách; vẽ top-50 bằng `drawMatches`
5. **So sánh trực tiếp:** chạy cả SIFT và ORB trên cùng cặp ảnh
— đếm số match đúng, đo thời gian thực thi
6. Thử ảnh thật (đọc từ file) thay vì ảnh tổng hợp